CS 301
High-Performance Computing

Assignment 08
Parallel Interpolation with Particle Mover using MPI

Khushi Pandya (202301406)
Aalok Thakkar (202301429)

April 29, 2026

Contents

1 Introduction

This assignment focuses on parallelizing the scatter interpolation and particle mover algorithm using MPI and OpenMP.

2 Theoretical Background

2.1 Bilinear Interpolation

Each particle contributes to four neighboring grid points using bilinear interpolation.

Let (x, y) lie inside a grid cell with corner (i, j):

$$w_{00} = (dx - l_x)(dy - l_y)$$
$$w_{10} = l_y(dx - l_x)$$
$$w_{01} = l_x(dy - l_y)$$
$$w_{11} = l_x l_y$$

The grid values are updated as:

$$F(i,j) += w_{00}, \quad F(i+1,j) += w_{10}$$
$$F(i,j+1) += w_{01}, \quad F(i+1,j+1) += w_{11}$$

2.2 Mover Equation

Grid values are interpolated back to particles:

$$F_i = \sum w \cdot F(X, Y)$$

Particle positions are updated as:

$$x_i^{new} = x_i + F_i \cdot \Delta x$$
$$y_i^{new} = y_i + F_i \cdot \Delta y$$

2.3 Computational Complexity

For N particles:

$$\text{Interpolation} = O(N)$$
$$\text{Mover} = O(N)$$

Total complexity:

$$O(N)$$

2.4 Parallel Computing Model

MPI:

- Distributes particles across processes
- Each process works on a subset of data
- Requires communication between processes

OpenMP:

- Shared memory parallelism
- Parallel loops using threads
- Requires synchronization (atomic/critical)

2.5 Amdahl's Law

$$S = \frac{1}{(1-P) + \frac{P}{N}}$$

Where:

- P = parallel fraction
- N = number of threads

This limits maximum achievable speedup.

2.6 Memory vs Compute Bound

For small particle sizes:

- Computation dominates $\rightarrow$ compute-bound

For large particle sizes:

- Memory access dominates $\rightarrow$ memory-bound

Interpolation phase becomes memory-bound due to frequent grid access.

3 Parallelization Strategy

- MPI distributes particles
- OpenMP parallelizes loops

4 Performance Metrics

$$S = \frac{T_2}{T_n}, \quad E = \frac{S}{N}$$

5 Results

5.1 Execution Time vs Threads

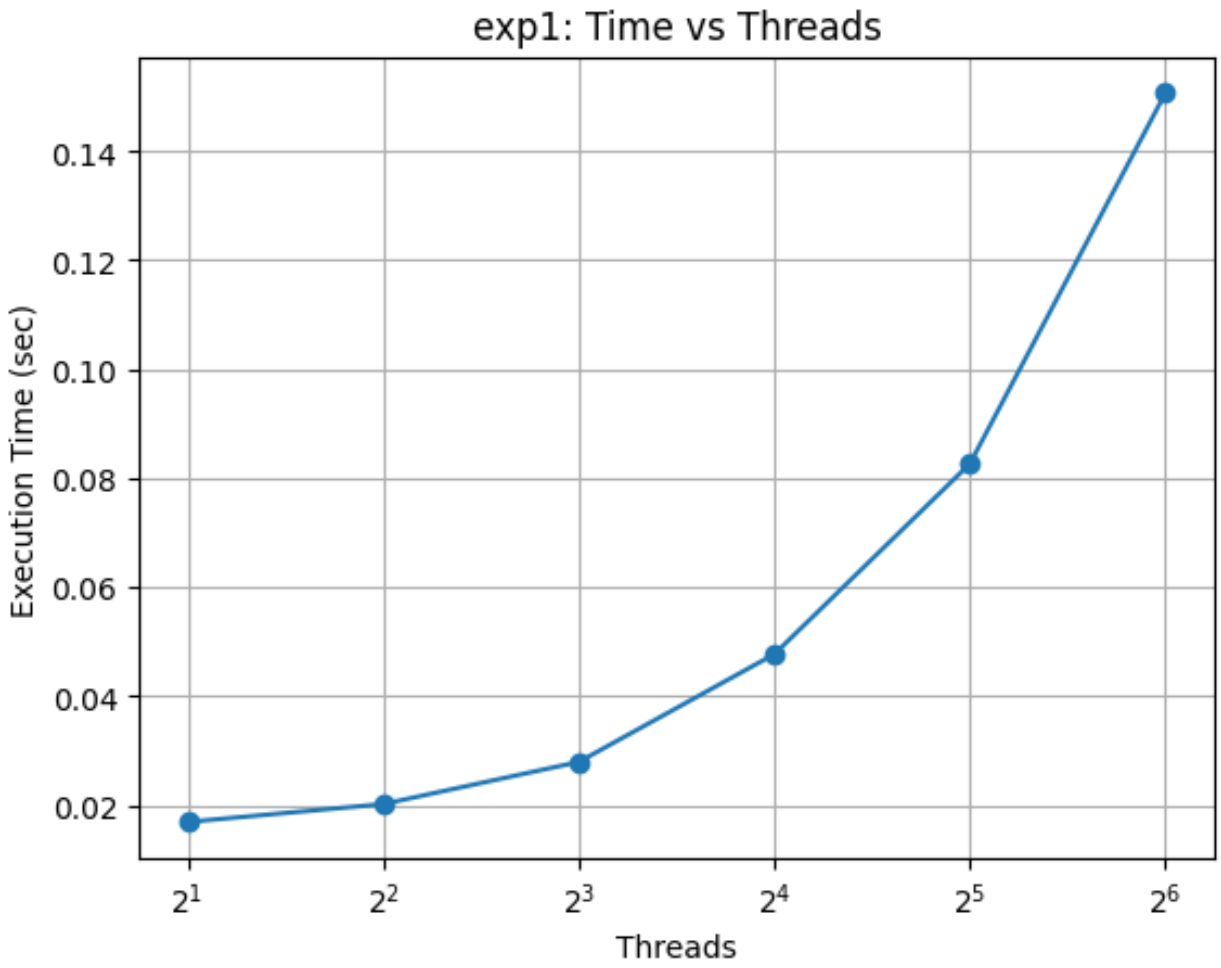

Figure 1: Experiment 1: Time vs Threads

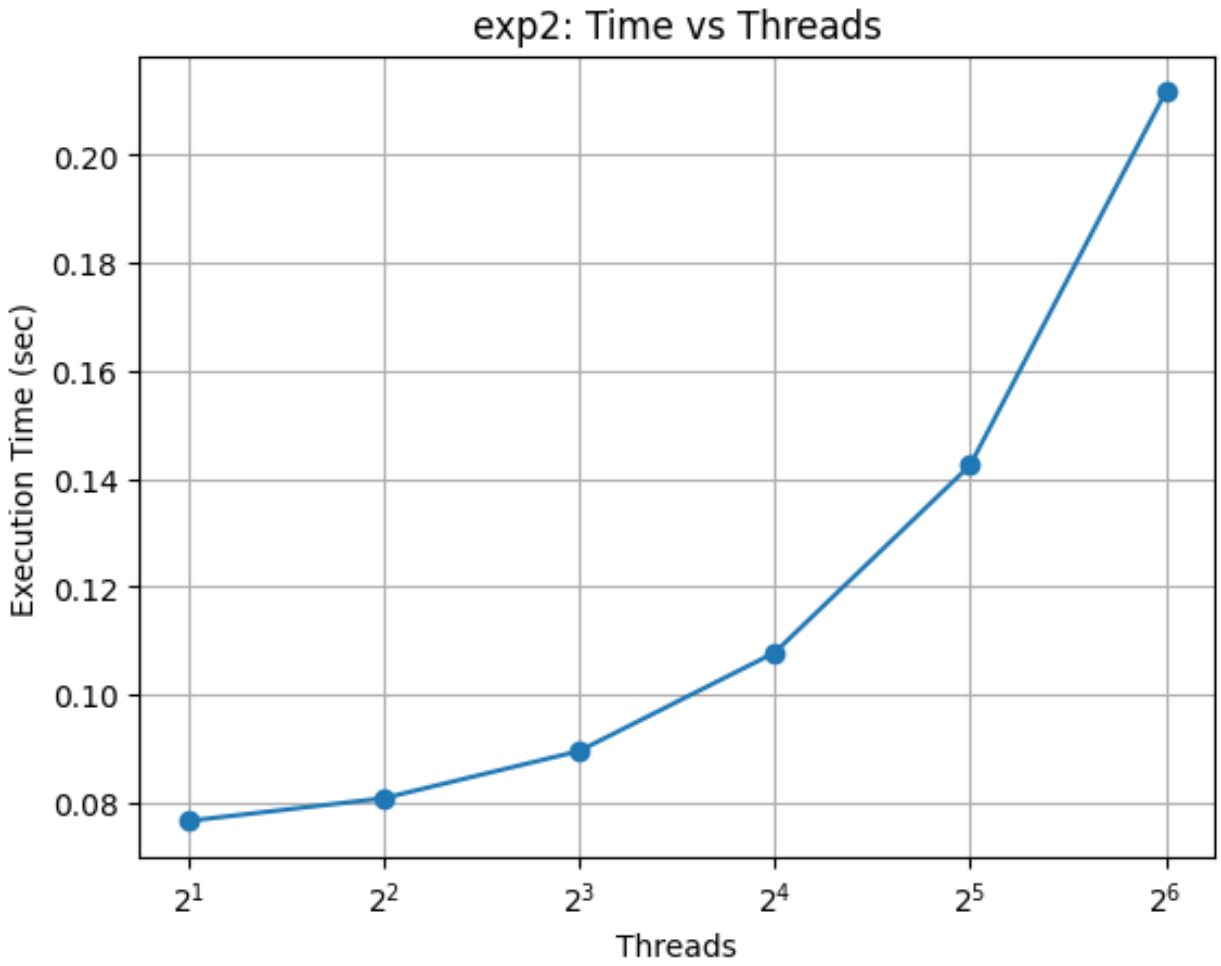

Figure 2: Experiment 2: Time vs Threads

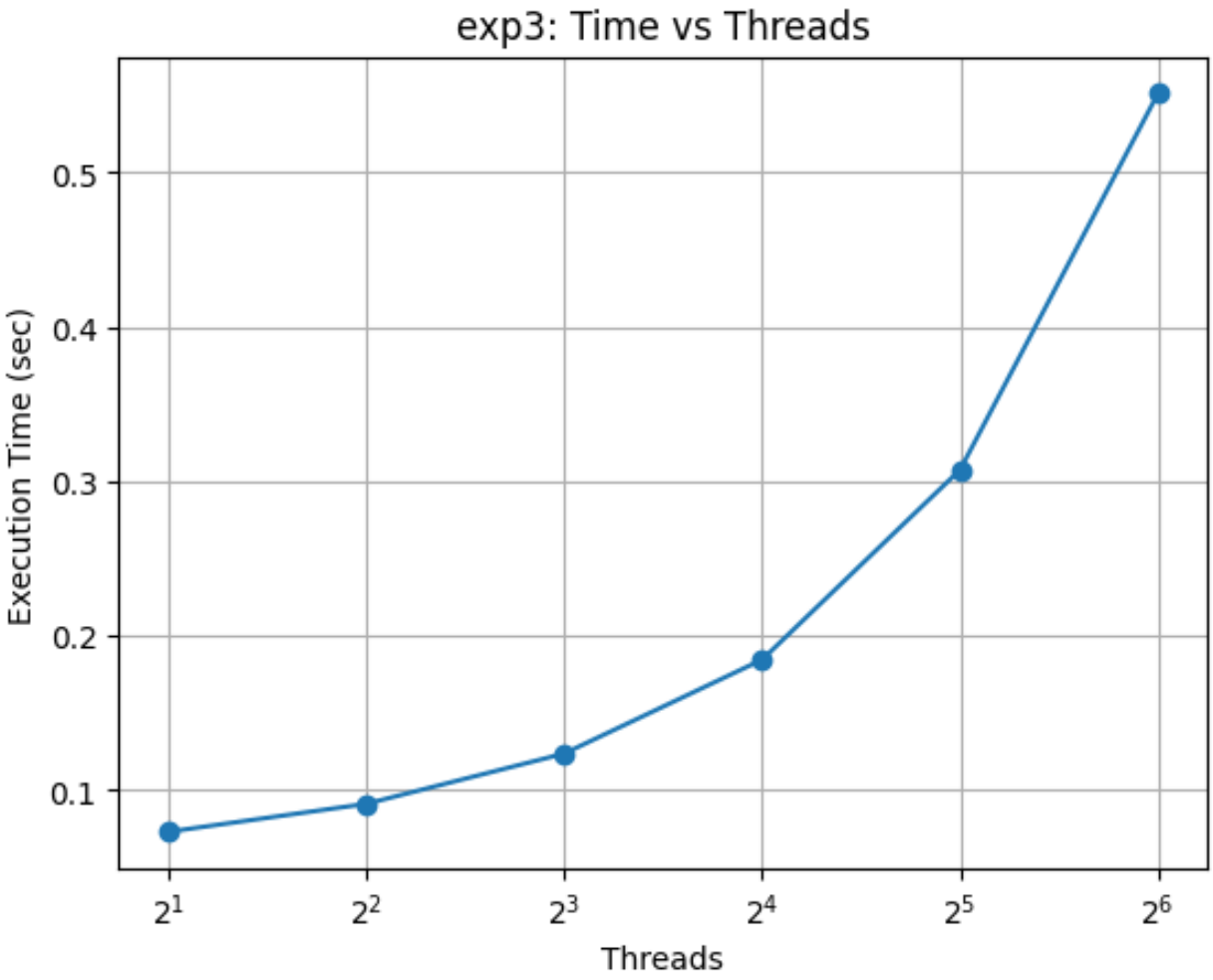

Figure 3: Experiment 3: Time vs Threads

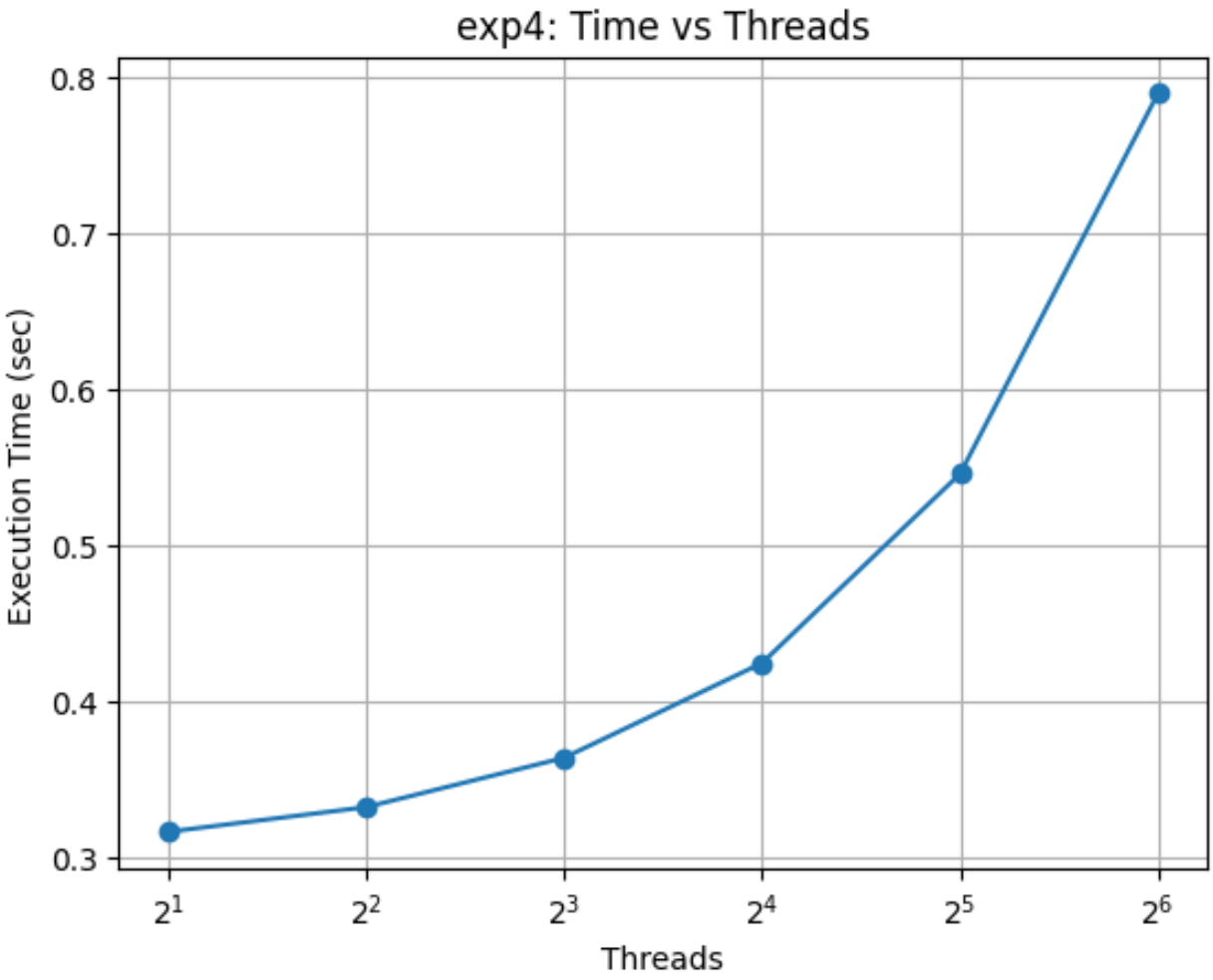

Figure 4: Experiment 4: Time vs Threads

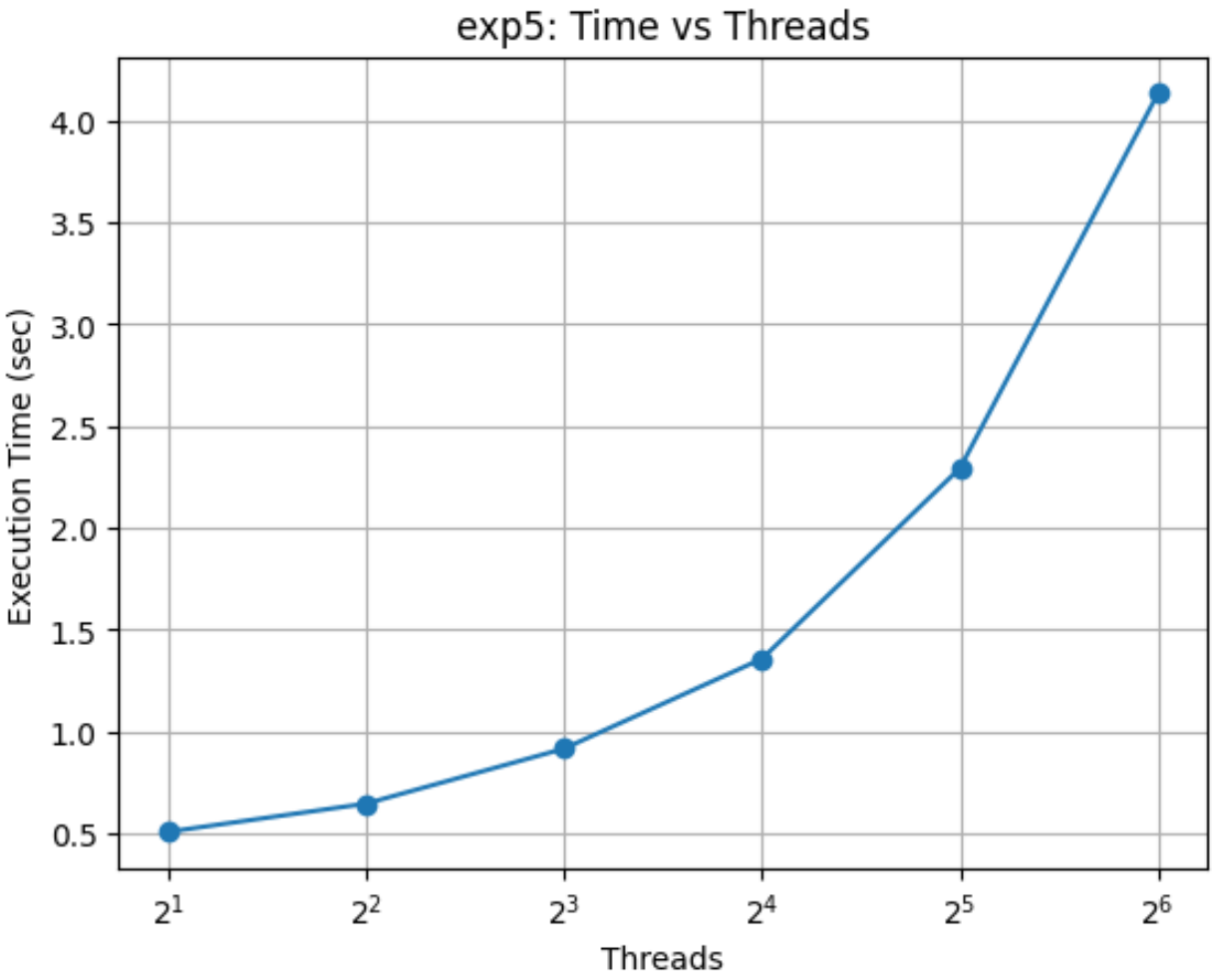

Figure 5: Experiment 5: Time vs Threads

5.2 Speedup vs Threads

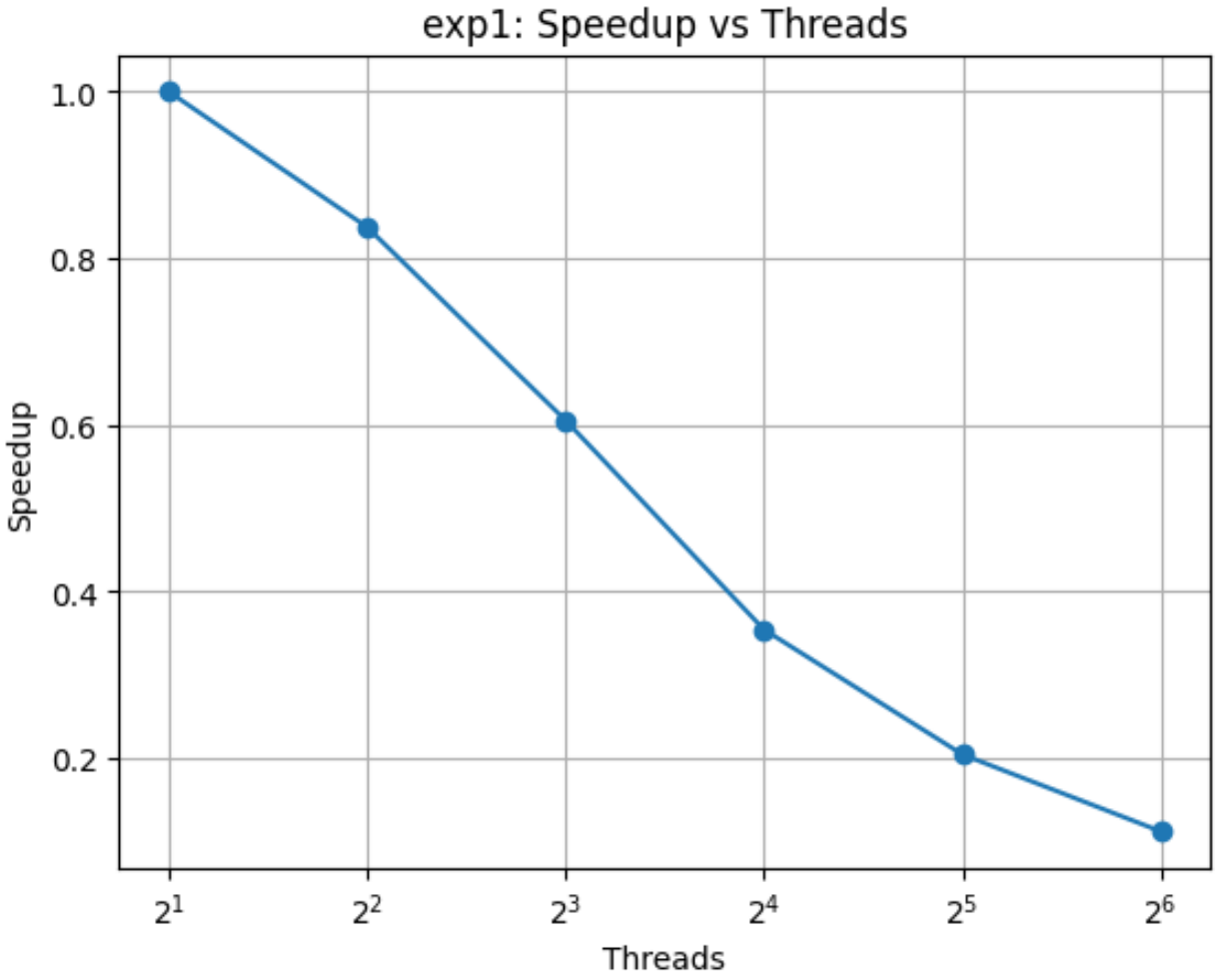

Figure 6: Experiment 1: Speedup

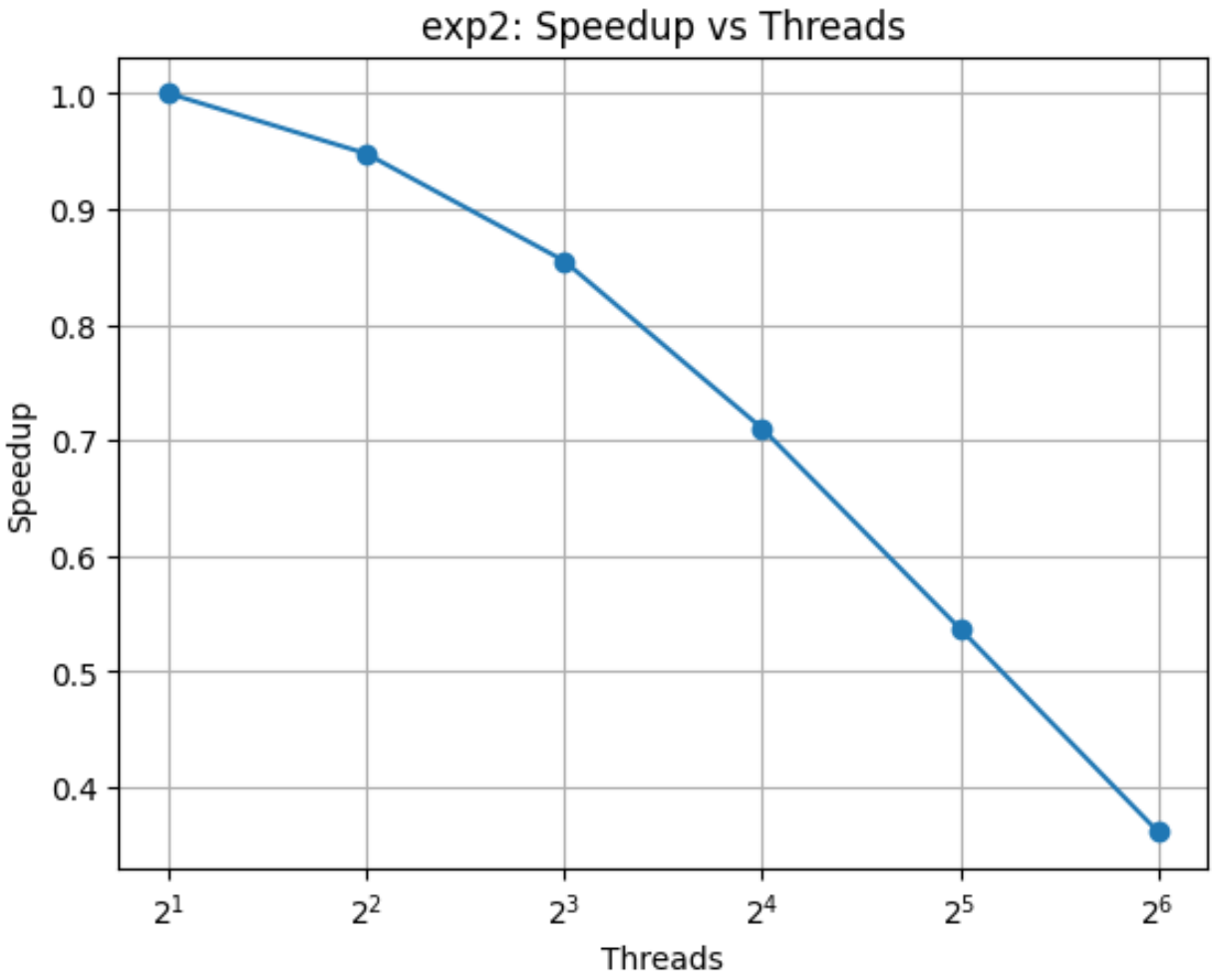

Figure 7: Experiment 2: Speedup

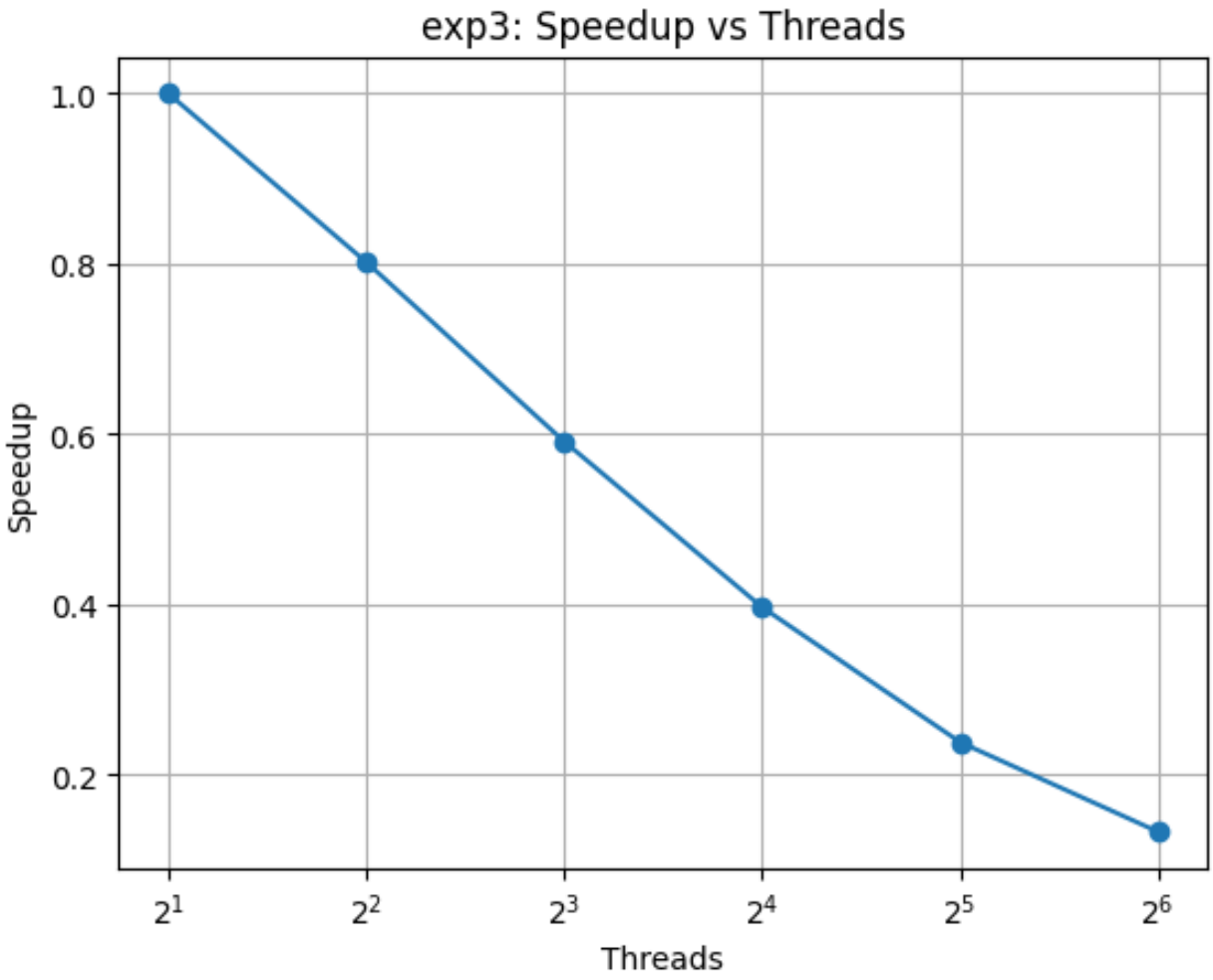

Figure 8: Experiment 3: Speedup

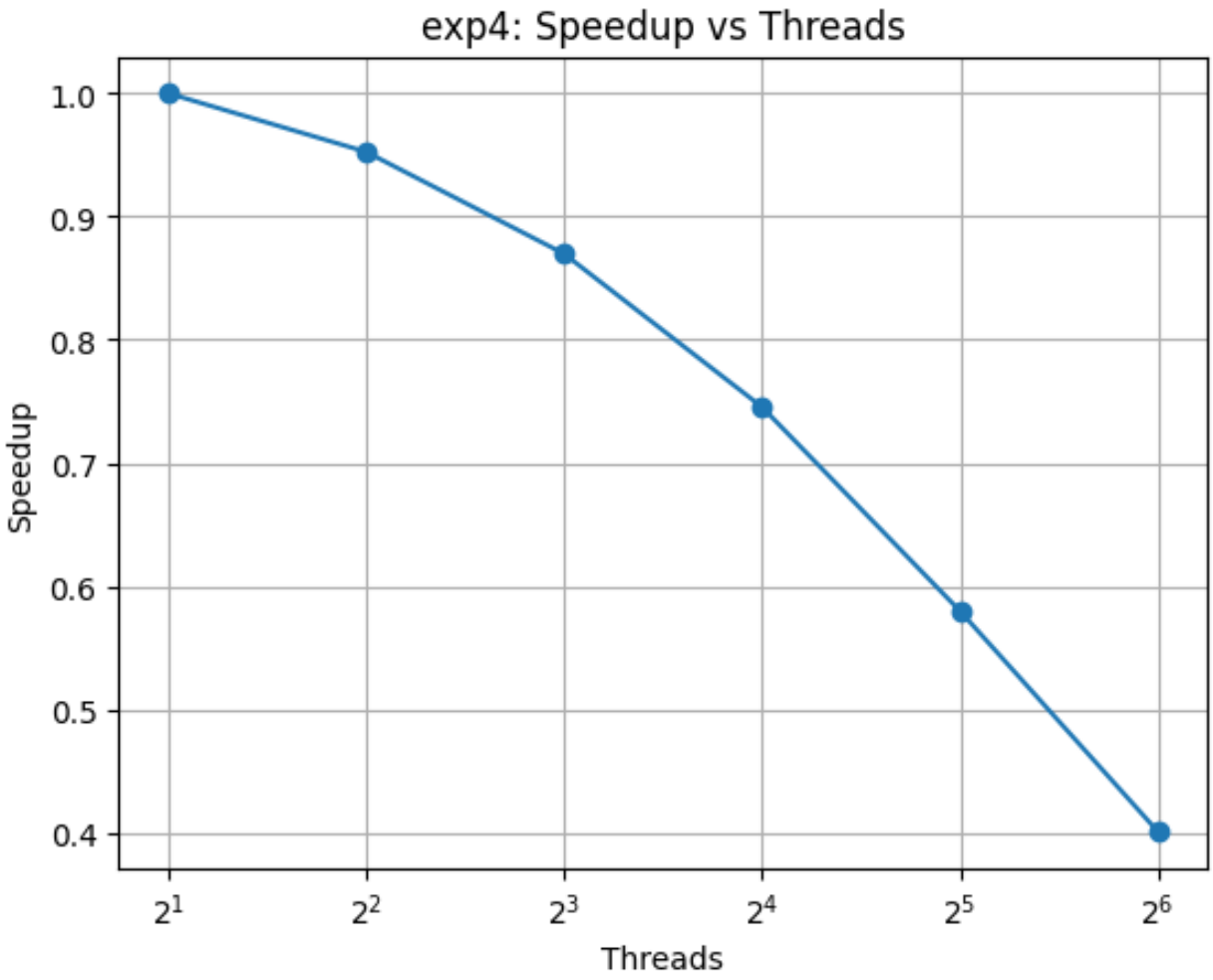

Figure 9: Experiment 4: Speedup

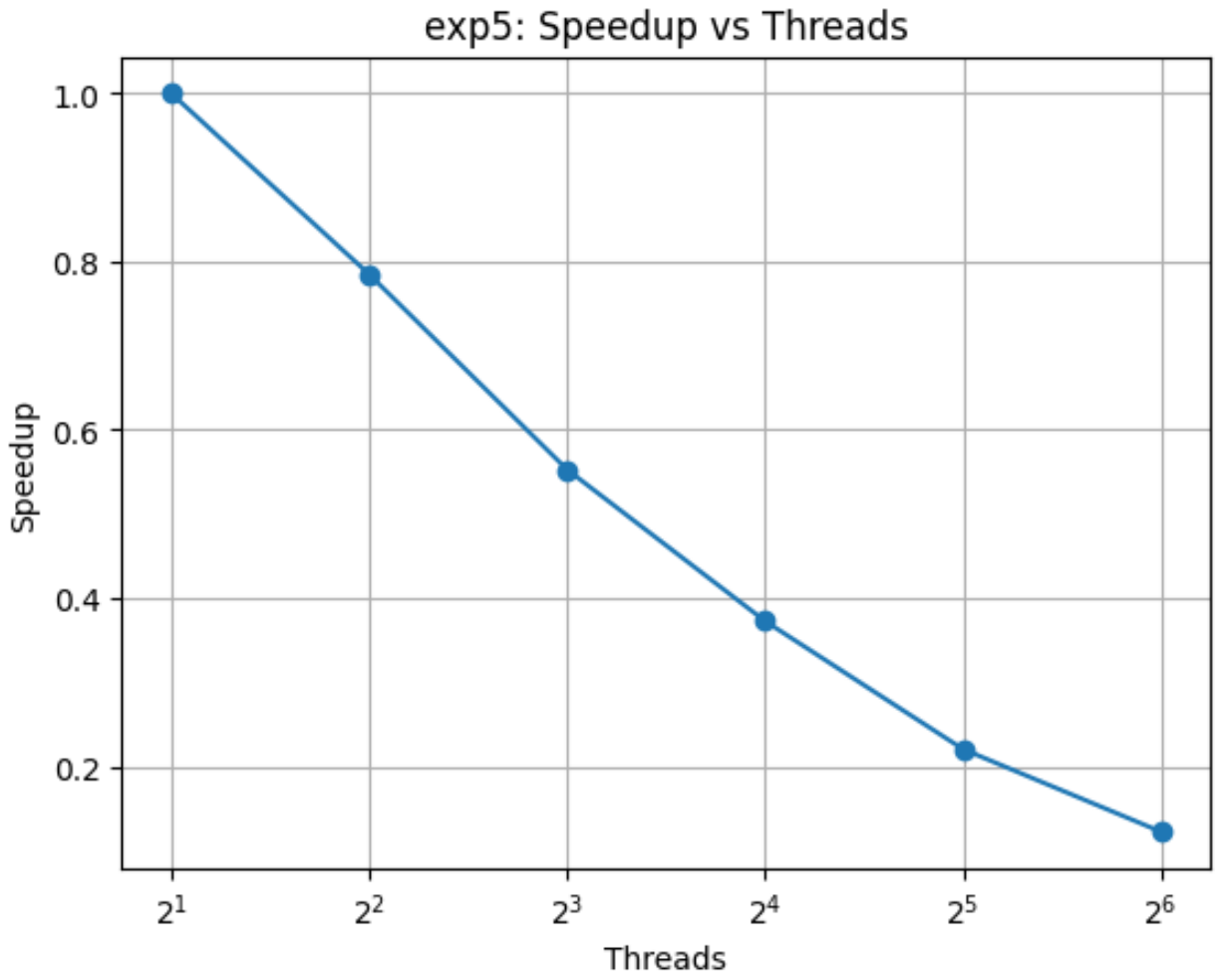

Figure 10: Experiment 5: Speedup

6 Analysis

6.1 Pseudocode

1. Initialize particles and grid
2. For each iteration:
   - Reset grid values
   - Perform interpolation (particle → grid)
   - Normalize grid values
   - Perform mover (grid → particle)
   - Update particle positions
3. Output final grid

6.2 Parallel Implementation

The algorithm is parallelized using a hybrid approach:

- MPI distributes particles across processes
- OpenMP parallelizes loops inside each process

Each process handles a subset of particles and computes interpolation and mover locally.

6.3 Race Conditions

Race conditions occur when multiple threads update the same grid cell during interpolation.

This is handled using:

- Atomic operations
- Synchronization mechanisms

However, these introduce overhead and reduce performance.

6.4 Execution Time Behavior

From the results, execution time increases as the number of threads increases.

Instead of decreasing, runtime becomes larger at higher thread counts.

This indicates poor scalability.

6.5 Speedup Analysis

Speedup is defined as:

$$S = \frac{T_2}{T_n}$$

Ideally, speedup should increase with threads.

However, in this case:

- Speedup decreases as threads increase
- Parallel overhead dominates computation

6.6 Parallel Efficiency

$$E = \frac{S}{N}$$

Parallel efficiency drops significantly with increasing threads, showing inefficient resource utilization.

6.7 Comparison with Serial Execution

The parallel implementation does not significantly outperform the serial version.

At higher thread counts, performance becomes worse than serial execution due to overhead.

6.8 Interpolation vs Mover Performance

Interpolation phase is the major bottleneck because:

- Multiple particles update the same grid points
- Requires synchronization
- Causes memory contention

Mover phase is faster because:

- Mostly read operations
- Less synchronization required

6.9 Scalability Analysis

The algorithm does not scale well with increasing threads.
Reasons include:

- MPI communication overhead
- Synchronization delays in OpenMP
- Load imbalance among processes
- Memory bandwidth limitations

6.10 Load Imbalance

As particles move, their distribution across processes becomes uneven.
Some processes handle more particles, leading to imbalance and idle time in others.

6.11 Memory vs Compute Bound

For small problem sizes:

- Computation dominates $\rightarrow$ compute-bound

For large problem sizes:

- Memory access dominates $\rightarrow$ memory-bound

Interpolation becomes memory-bound due to frequent grid updates.

6.12 Amdahl's Law Interpretation

$$S = \frac{1}{(1-P) + \frac{P}{N}}$$

Even with large number of threads, speedup is limited by the serial portion and overhead.
Thus, maximum achievable performance is bounded.

6.13 Reasons for Poor Performance

- High MPI communication overhead
- Synchronization during grid updates
- Cache conflicts and memory contention
- Small workload per thread

6.14 Optimization Techniques

Performance can be improved using:

- Grid privatization (local grids per thread)
- Reduction-based updates
- Domain decomposition
- Better memory layout (cache-friendly)

6.15 If Unlimited Resources Are Available

- Use GPU acceleration for interpolation
- Use distributed memory optimization
- Reduce communication latency

6.16 Final Observation

The implementation is functionally correct but not performance-efficient.
The interpolation phase dominates runtime, and overhead limits scalability.
Further optimization is required to achieve ideal parallel speedup.

7 Conclusion

The implementation does not scale efficiently due to overhead.